

INFORME DE REFLEXIÓN TÉCNICA

Práctico N° 5 - De Texto Caótico a Dataset Limpio

1. Estructuras de Datos: Lista de Diccionarios vs Lista de Listas

Elegí usar una **lista de diccionarios** en lugar de una lista de listas por las siguientes razones:

Legibilidad: Con diccionarios, accedo a los datos usando claves descriptivas como `cliente["email"]` en lugar de índices numéricos como `cliente[2]`. Esto hace el código autodocumentado y reduce errores.

Acceso a datos: Los diccionarios permiten acceder directamente a cualquier campo sin recordar posiciones. Además, facilitan agregar nuevos campos sin modificar código existente.

Compatibilidad: Esta estructura se convierte directamente a JSON y CSV usando las bibliotecas estándar de Python.

2. Lógica Regex: Validación del Teléfono

Para normalizar teléfonos utilicé un enfoque en dos pasos:

Paso 1 - Extracción: Usé `re.findall(r'\d', linea)` para extraer TODOS los dígitos de la línea, ignorando guiones, espacios, paréntesis u otros caracteres.

Paso 2 - Validación: Verifico que el resultado tenga exactamente 10 dígitos. Si hay más, tomo los últimos 10 (descartando posibles IDs al inicio). Si hay menos, el teléfono es inválido.

Este enfoque es robusto porque funciona con cualquier formato de entrada ("351-456-7890", "(351) 555 1234", "351.444.1111").

3. Gestión de Errores y Datos Faltantes

Apliqué la siguiente estrategia ante datos problemáticos:

Líneas de sistema: *Ignoradas.* Las líneas que comienzan con "####", separadores o errores se detectan y no se procesan.

Email faltante o inválido: *Descarte.* El email es el identificador único, por lo que registros sin email válido (como "robertog@gmail" sin extensión) se descartan.

Teléfono faltante: *Marcación.* Se asigna "Desconocido" porque el cliente aún puede ser contactado por email.

Emails duplicados: *Conservar el primero.* Se registra el duplicado en el log.

4. Escalabilidad: Procesando 1 Millón de Filas

Para manejar archivos masivos, implementaría los siguientes cambios:

Lectura por chunks: En lugar de cargar todo con `readlines()`, usaría un iterador que procese línea por línea, reduciendo el uso de memoria.

Formato Parquet: Para la persistencia, reemplazaría CSV/JSON por Parquet, que ofrece compresión columnar eficiente y es el estándar en Big Data.

Procesamiento con Pandas/Dask: Pandas permite leer en chunks (`pd.read_csv(chunksize=10000)`), mientras que Dask permite procesamiento paralelo manteniendo una API similar.